Лабораторна робота №5

Тема: Розробка клієнтської частини (Front-end) та інтеграція з API

Студент: Бабенко Олександр Олександрович

Група: 402-ТН

Викладач: Тютюнник Петро Богданович

Посилання на Git-репозиторій:

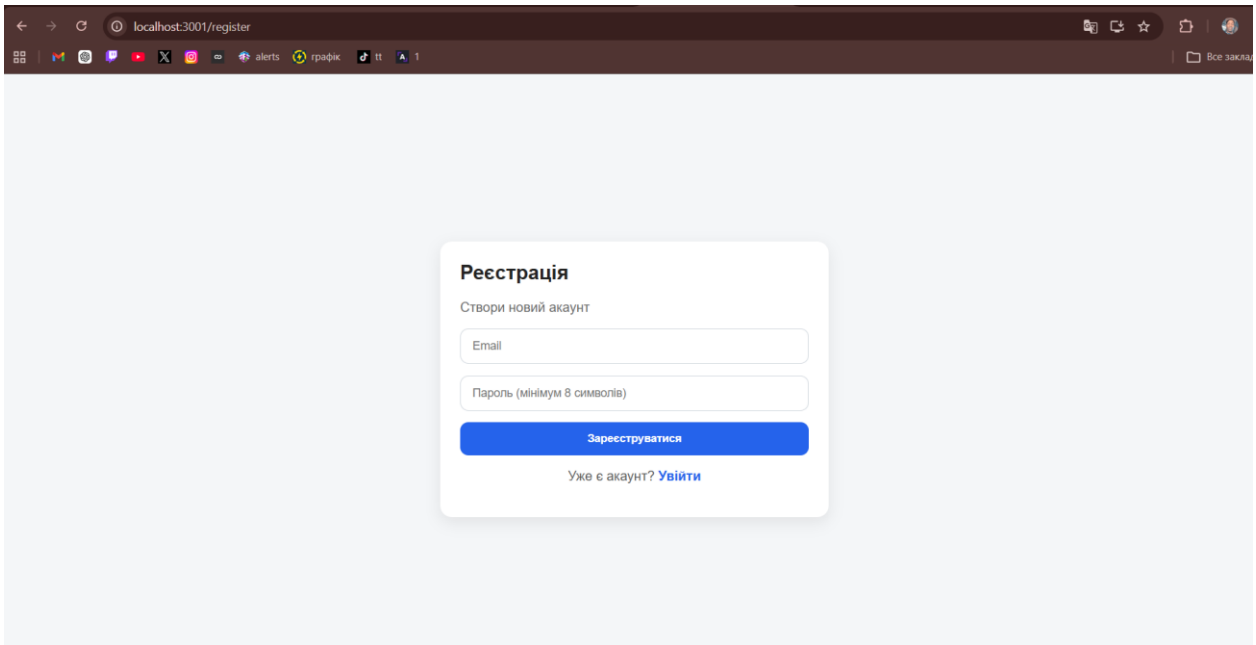
https://github.com/Radonskiy/TRVD_2026_402TN_Babenko_Labs

У ході виконання лабораторної роботи було розроблено клієнтський додаток із використанням React та інтегровано його із бекенд API, створеним у попередніх лабораторних роботах.

Посилання на відео-демонстрацію:

https://drive.google.com/file/d/12JZGN2-tcXPt6UkgMfGNLPboGnCMO8hq/view?usp=sharing

Виконання



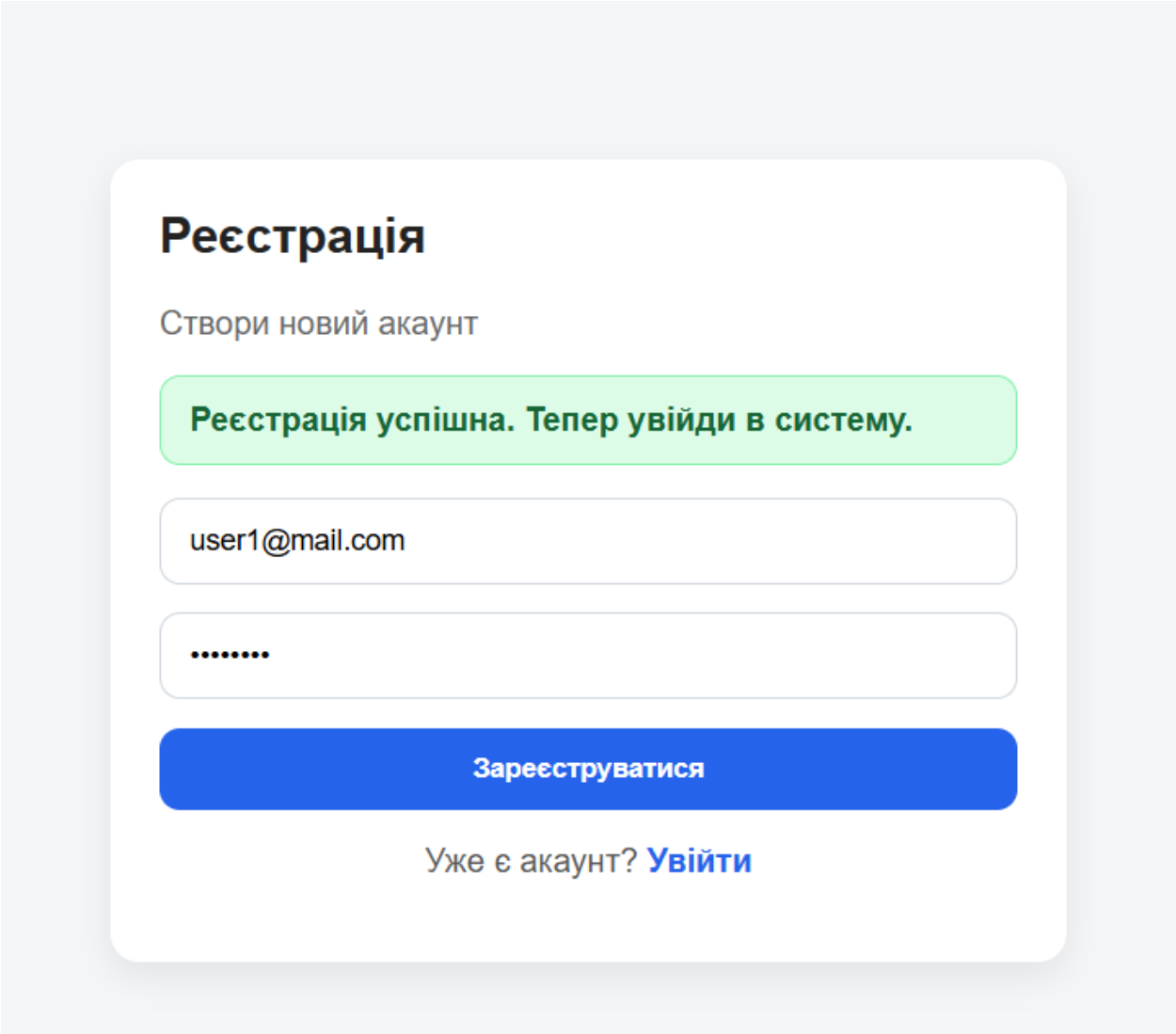


Рисунок 1,2 – Форма реєстрації користувача

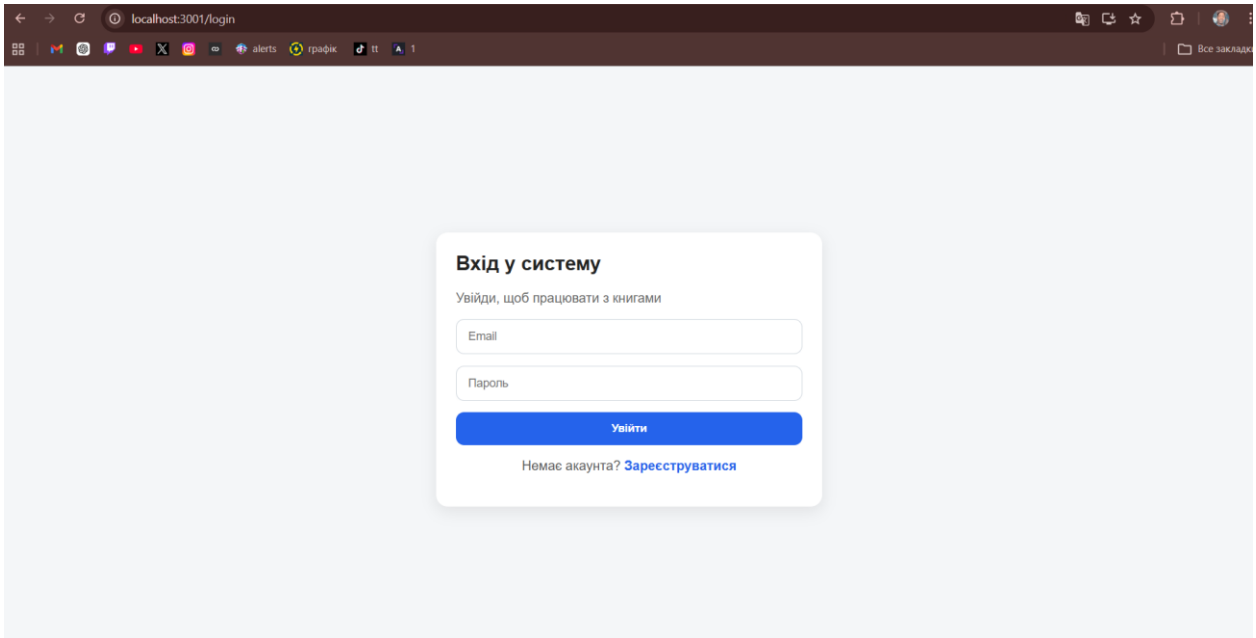


Рисунок 3 – Форма логіну користувача

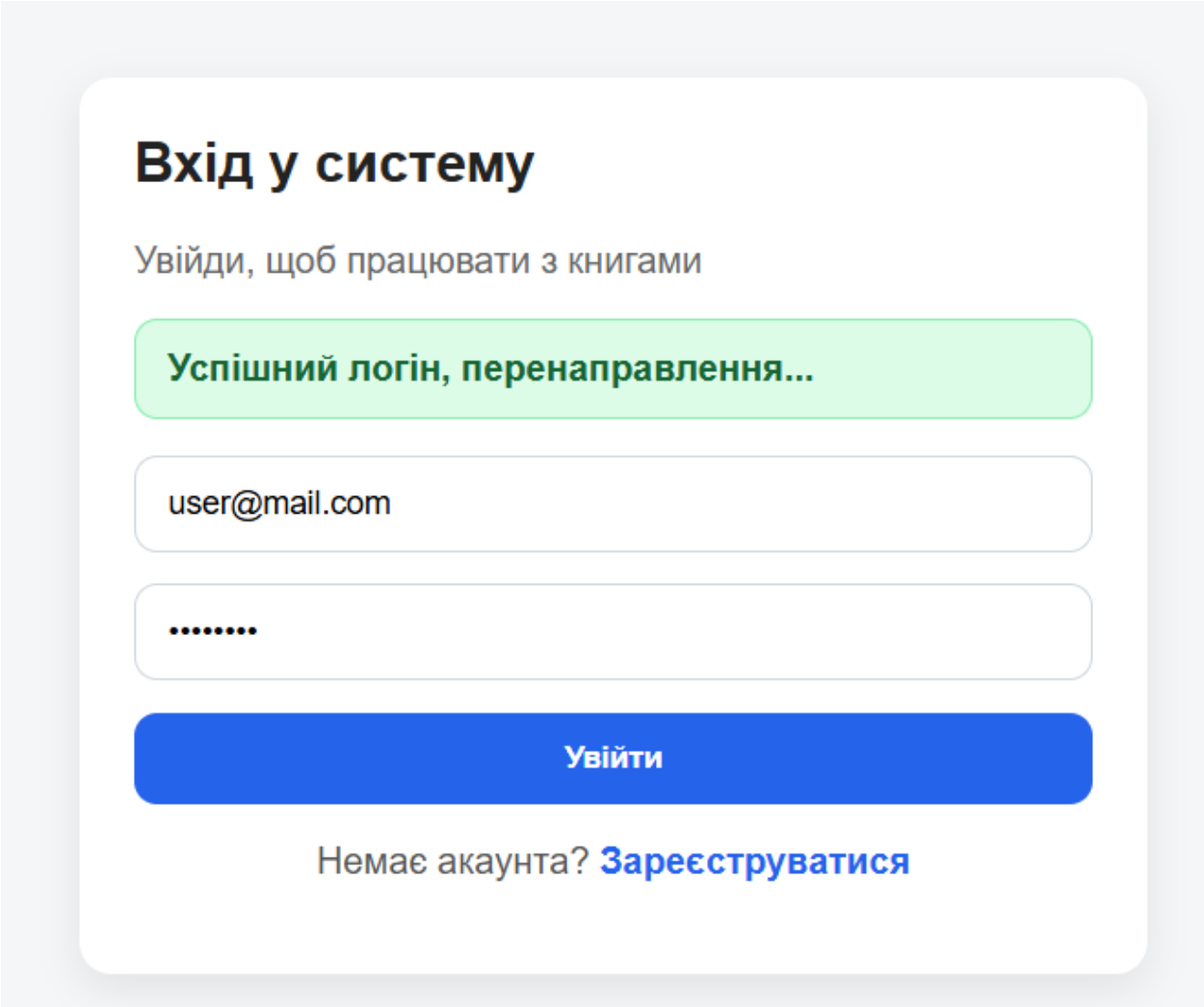


Рисунок 4 – Успішний логін користувача

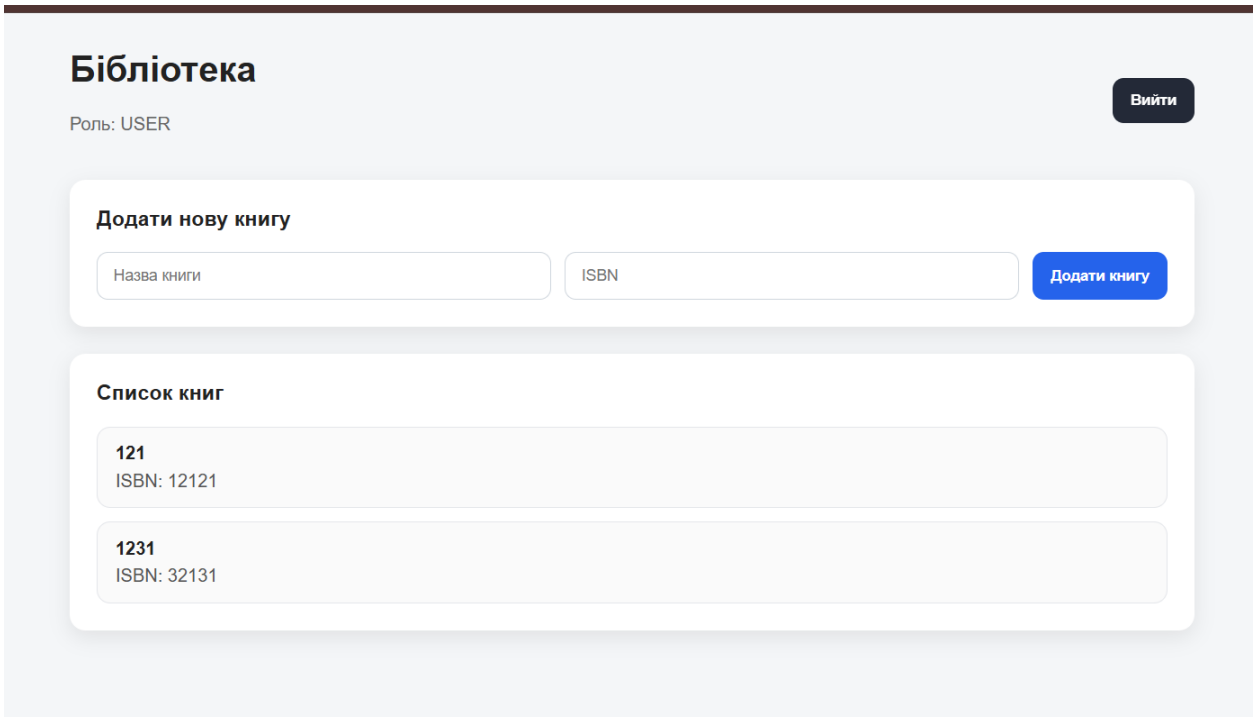


Рисунок 5 – Успішний логін користувача та перехід на захищену сторінку

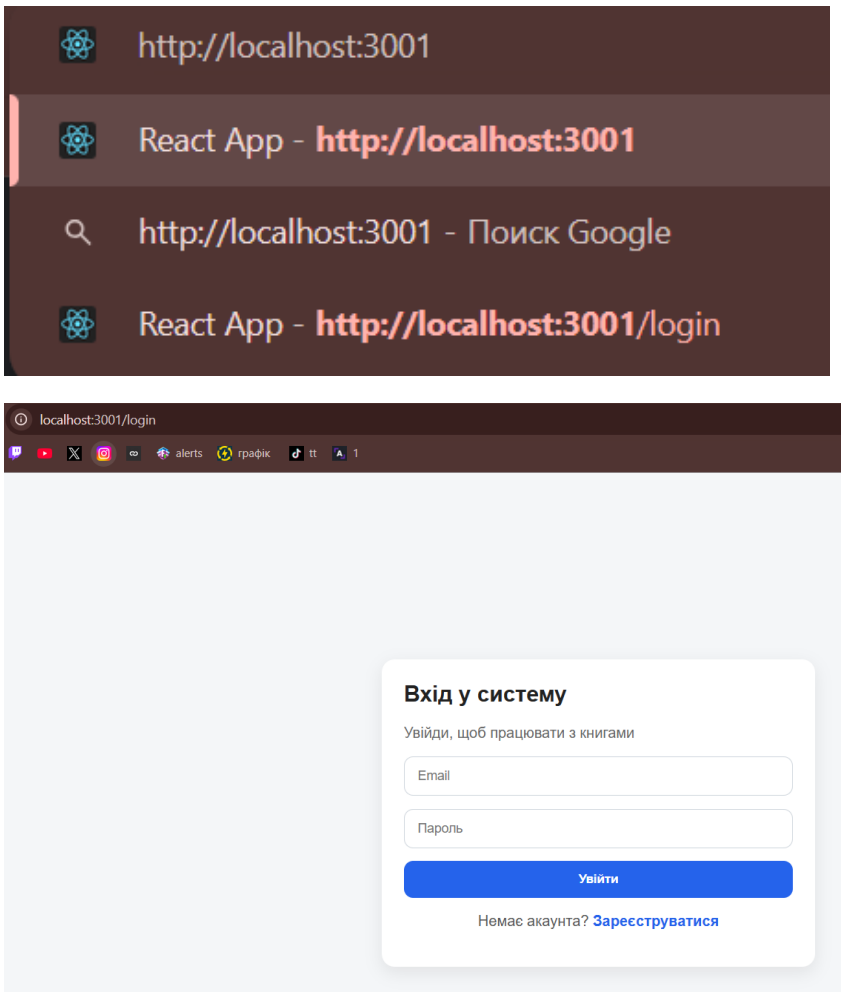
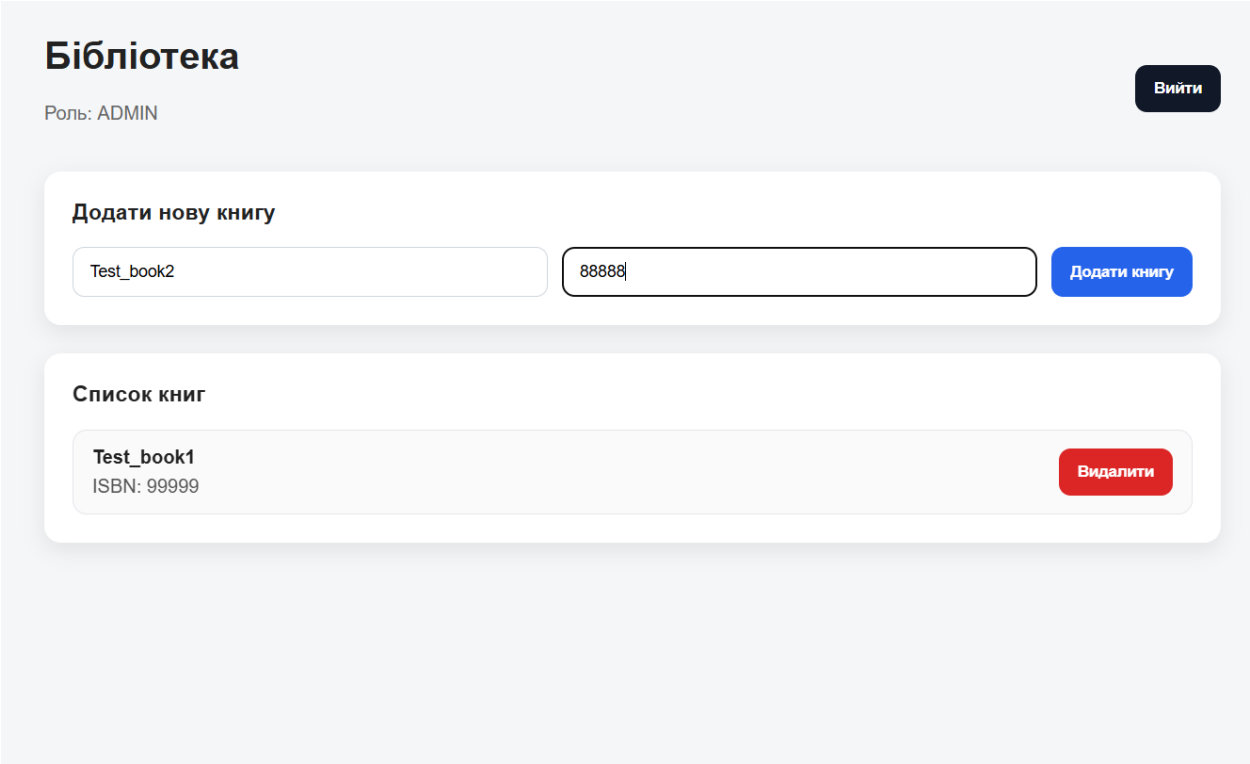


Рисунок 6 – Демонстрація Protected Route (перенаправлення на сторінку логіну без авторизації)



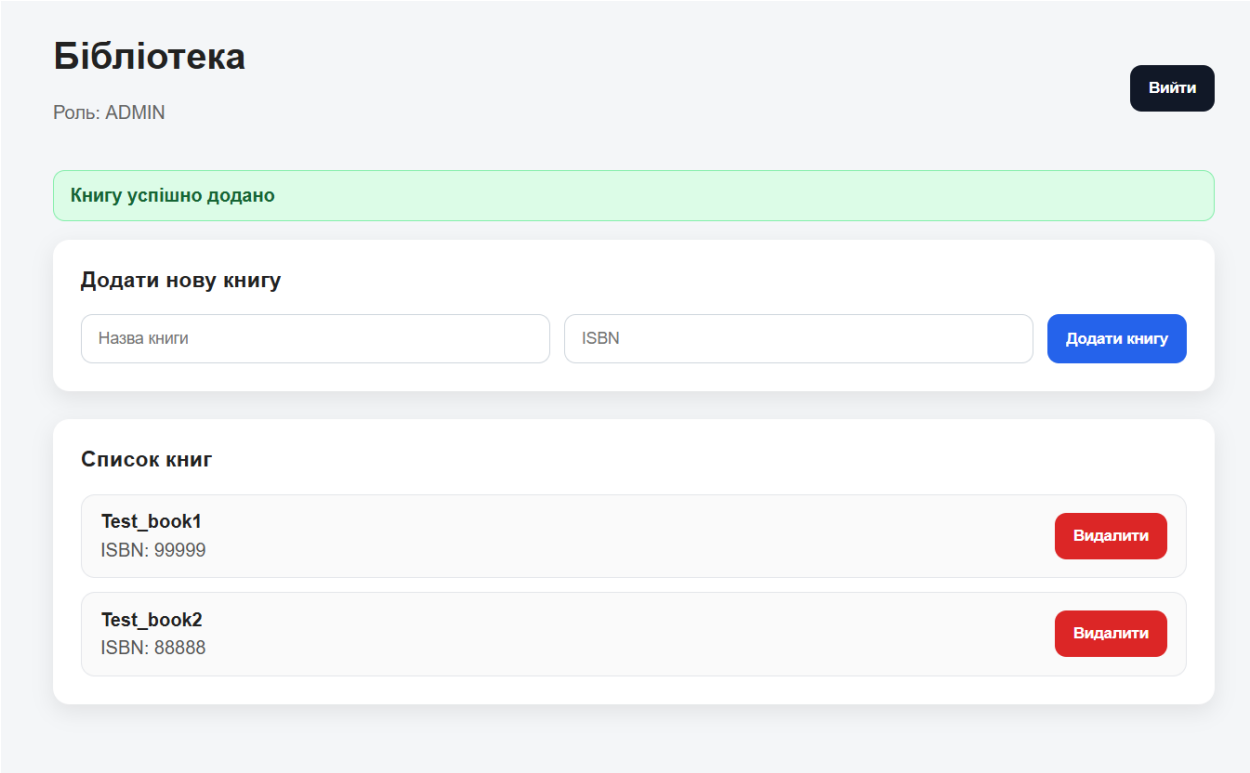


Рисунок 7 – Додавання нової книги (Create)

Лише користувач з АДМІНом може видаляти книги

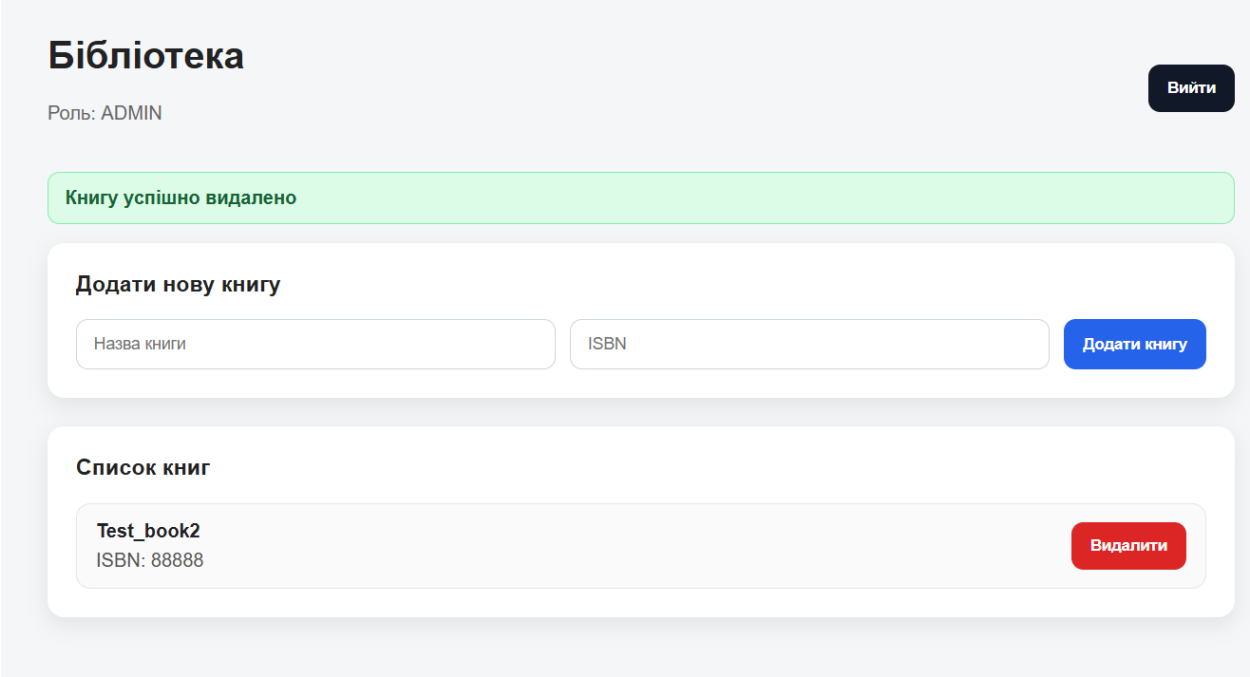


Рисунок 8 – Вхід з роллю Admin (кнопка Delete є)

Тоді як у звичайного користувача навіть кнопки «Видалити» немає

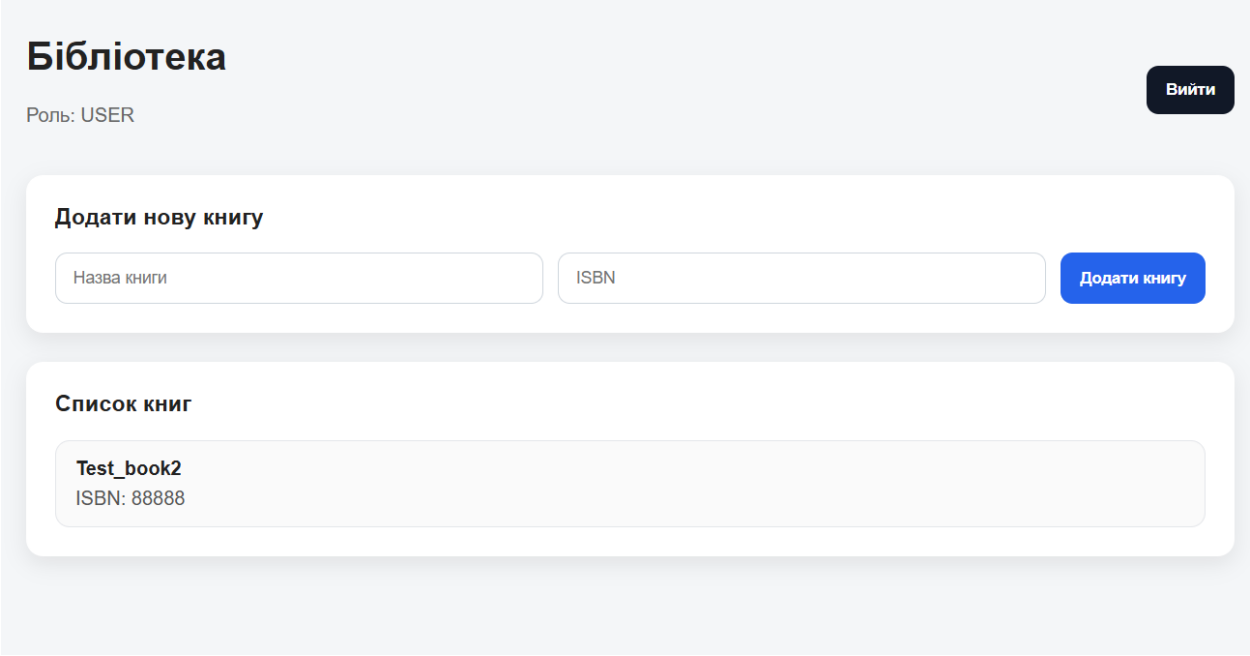


Рисунок 9 – Вхід з роллю User (кнопки Delete немає)

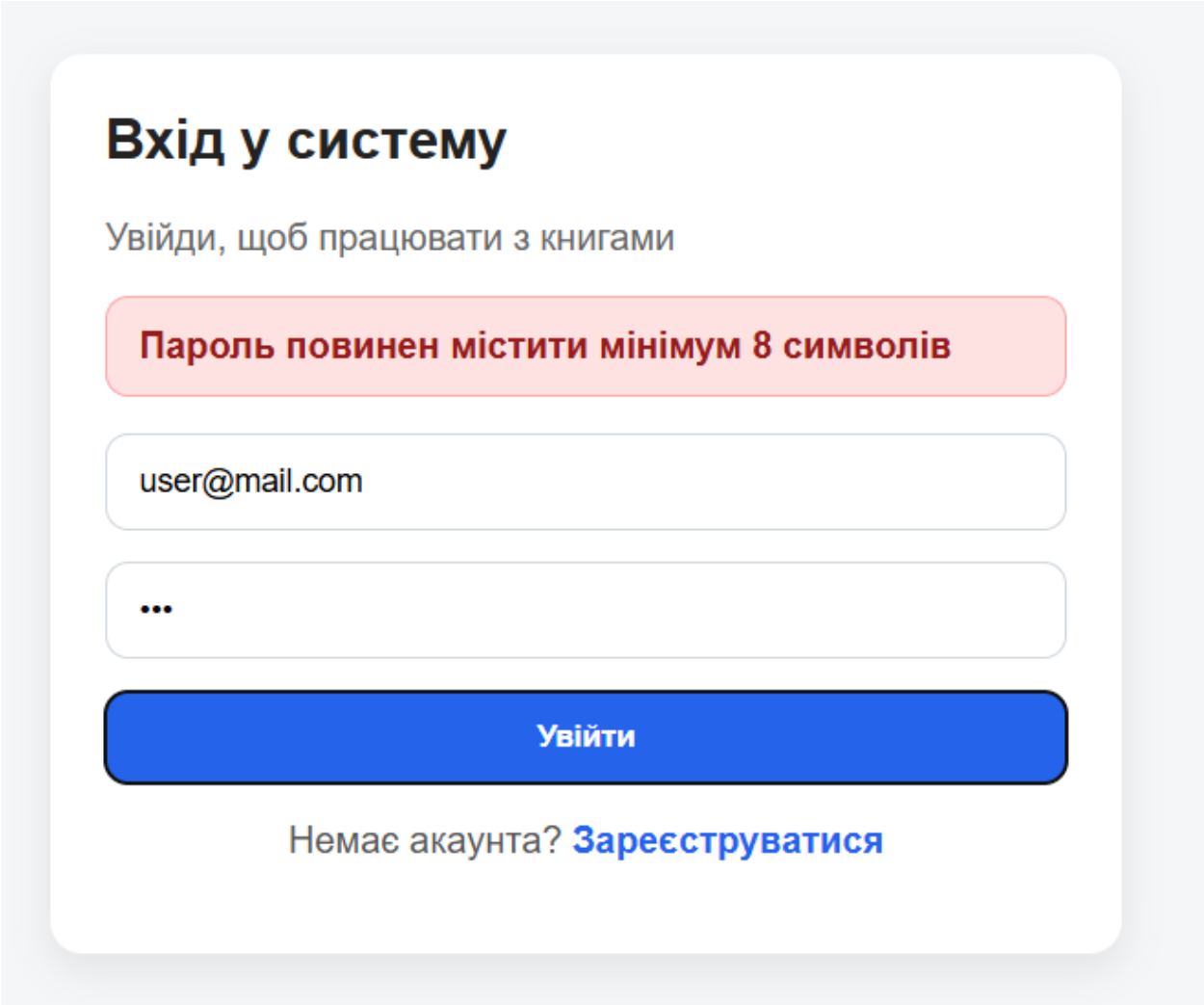


Рисунок 10 – Помилка якщо пароль містить менше 8 символів

Контрольні запитання

1. Що таке SPA (Single Page Application) і чим воно відрізняється від класичних MPA? Які переваги та недоліки?

SPA (Single Page Application) — це односторінковий веб-додаток, у якому основна сторінка завантажується один раз, а подальша навігація та оновлення інтерфейсу виконуються динамічно за допомогою JavaScript без повного перезавантаження сторінки.

MPA (Multi Page Application) — це багатосторінковий додаток, у якому кожен перехід між сторінками викликає новий запит до сервера та повне перезавантаження сторінки.

Переваги SPA:

- швидка робота після першого завантаження;
- кращий користувацький досвід;
- зручна взаємодія з API.

Недоліки SPA:

- складніша логіка розробки;
- складніше забезпечити SEO;
- більше навантаження на браузер.

---

2. Що таке Virtual DOM (у контексті React/Vue) і навіщо він потрібен?

Virtual DOM — це віртуальне представлення реального DOM у пам’яті браузера. Коли стан компонента змінюється, React спочатку оновлює Virtual DOM, а потім порівнює його з попередньою версією. Після цього в реальний DOM вносяться лише ті зміни, які справді потрібні.

Це потрібно для оптимізації продуктивності, оскільки прямі часті зміни реального DOM є повільнішими.

---

3. Як працює Routing на клієнті? Чим History API відрізняється від звичайних посилань?

Клієнтський routing дозволяє перемикатися між сторінками без повного перезавантаження браузера. У React для цього часто використовується бібліотека React Router.

History API дозволяє змінювати URL адресний рядок браузера без перезавантаження сторінки. Це дає можливість створювати відчуття переходу між сторінками, хоча фактично працює один SPA-додаток.

Звичайні HTML-посилання викликають повне перезавантаження сторінки, а History API — ні.

---

4. Де безпечніше зберігати JWT токени на клієнті: localStorage, sessionStorage чи HttpOnly Cookies? Обґрунтуйте.

Найбезпечнішим варіантом вважаються HttpOnly Cookies, оскільки вони недоступні з JavaScript і краще захищені від XSS-атак.

localStorage і sessionStorage простіші у використанні, але токени в них доступні JavaScript-коду, тому при наявності XSS-уразливості їх можна викрасти.

У навчальних проєктах часто використовують localStorage через простоту реалізації, але в реальних системах перевага надається HttpOnly Cookies.

---

5. Що таке CORS? Чому браузер блокує запити до іншого домену і як це вирішити?

CORS (Cross-Origin Resource Sharing) — це механізм безпеки браузера, який контролює виконання запитів між різними джерелами (origin), наприклад між localhost:3001 і localhost:3000.

Браузер блокує такі запити, щоб запобігти несанкціонованому доступу до ресурсів іншого домену.

Проблема вирішується налаштуванням CORS на сервері, тобто сервер повинен явно дозволити запити з потрібного джерела.

---

6. Поясніть концепцію State Management (управління станом). Коли достатньо локального стану компонента, а коли потрібен глобальний (Redux/Context/Pinia)?

State Management — це керування даними, які змінюються під час роботи додатку і впливають на відображення інтерфейсу.

Локальний стан використовується тоді, коли дані потрібні лише одному компоненту або невеликій його частині. Наприклад, значення полів форми або стан модального вікна.

Глобальний стан потрібен тоді, коли одні й ті самі дані використовуються в багатьох компонентах. Наприклад, інформація про поточного користувача, токен, кошик покупок або загальні налаштування додатку.

Для глобального стану застосовують Context API, Redux, Pinia та інші інструменти.

Висновок

У ході виконання лабораторної роботи було розроблено клієнтську частину веб-додатку з використанням React та інтегровано її із серверним API, реалізованим у попередніх лабораторних роботах.

Було налаштовано структуру фронтенд-проєкту, створено окремі сервіси для роботи з API та реалізовано механізм автоматичного додавання JWT токена до HTTP-запитів через interceptor. Також було реалізовано обробку помилок 401 Unauthorized.

У додатку реалізовано форму логіну, захищені маршрути та інтерфейс для роботи з сутністю «Книга». Користувач може переглядати список книг, додавати нові записи та видаляти їх через зручний інтерфейс.

Таким чином, мету лабораторної роботи досягнуто: створено клієнтський SPA-додаток, що взаємодіє із захищеним API та забезпечує виконання основних CRUD-операцій.